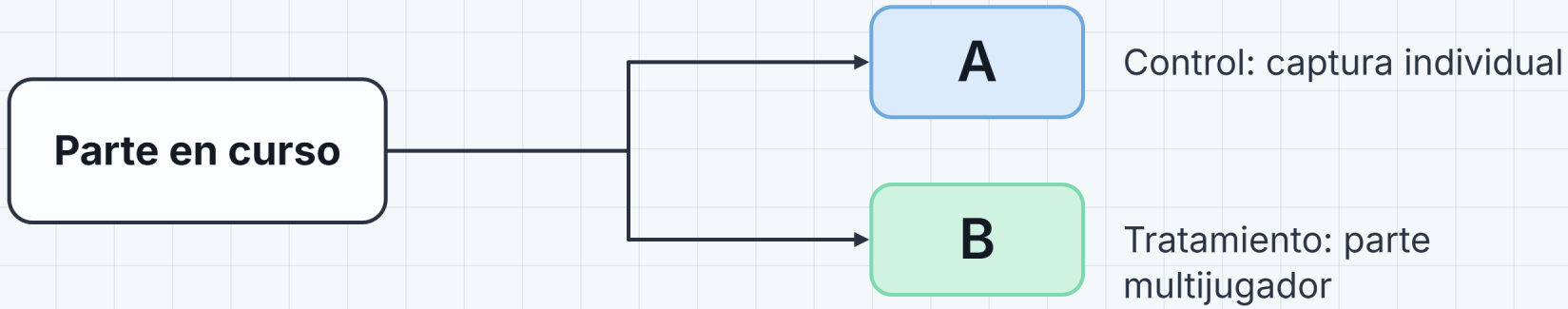
INGENIERÍA DE PRODUCTO

El Prototipo como Experimento Científico

De bocetos estáticos a despliegues en producción usando el método científico.

Equipo: Ahhmed Affif Jalife Burgueño · Elias Tamayo Salcedo · Verónica Acevedo Carrillo
Josselyn Alexa Rivera Chávez · Arturo Rafael Cornejo Escobar

Caso: parte multijugador del Funky Prototype · Bomberos de Ensenada



El Mito

Un prototipo es un dibujo estático en Figma, código desechable o una maqueta para «ver cómo se ve».



El Nuevo Paradigma

Un prototipo es un Cambio de Producto Testeable. Es código real, lanzado a una porción de usuarios reales, diseñado para medir su impacto de forma matemática y científica.





NUESTRO CASO

Nuestro Funky Prototype (el parte multijugador) es una hipótesis: si la unidad llena el parte entre varios, el tiempo activo para enviarlo baja frente a que lo capture una sola persona.

Prototipado Clásico vs. en Producción

	Clásico (El Pasado)	Producción (El Presente)
Entorno	Figma, Sketch o entorno local. Nuestro Funky: sin servidor.	Servidores en vivo con bases de datos y APIs reales: nuestro hito 4.
Usuarios	Evaluadores reclutados, ambiente controlado y sesgado.	Tráfico real que se comporta natural, sin saber que lo evalúan: partes reales.
Gestión de Riesgo	Cero riesgo técnico, alto riesgo de negocio (lanzar algo que nadie quiere).	Riesgo controlado con Feature Flags (despliegue condicional).
Tipo de Datos	Opiniones («me gusta», «no me gusta»).	Datos cuantitativos con significancia estadística (Pruebas A/B).

“Diseña como si tuvieras razón, prueba como si estuvieras equivocado.”

— Erin Weigel

Fase 1: Ejecución

Diseña como si tuvieras razón.

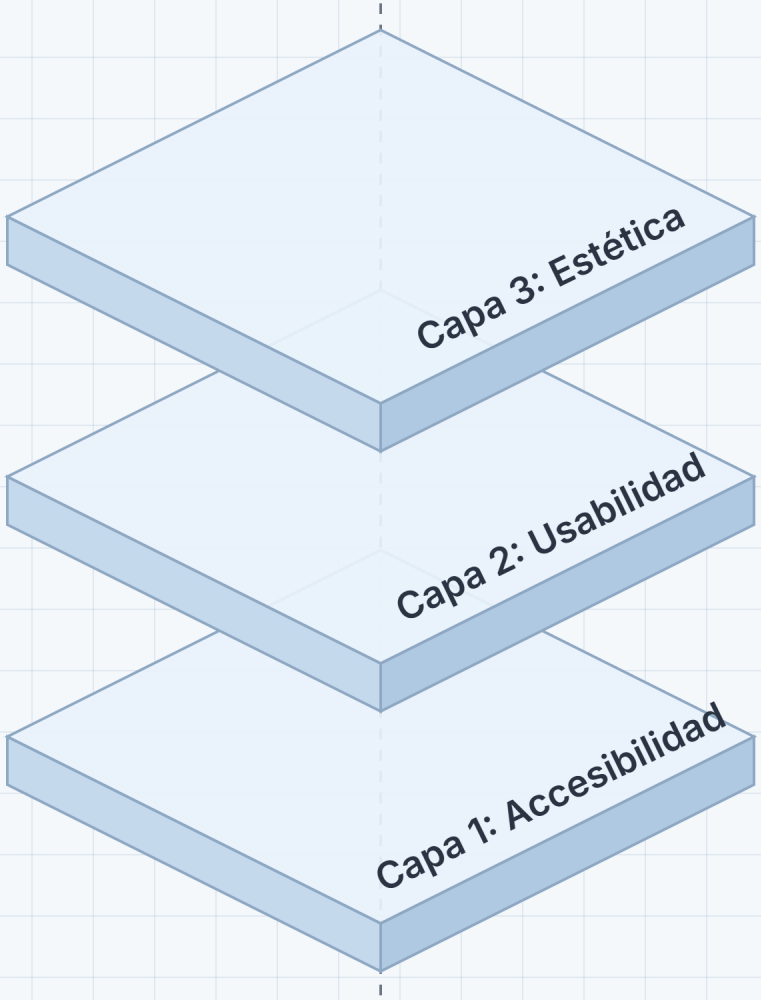
Construye la solución con el máximo rigor técnico y de UX. Si la ejecución falla, el experimento es basura: aísla la variable de errores.

Fase 2: Validación

Prueba como si estuvieras equivocado.

Adopta un escepticismo radical. Asume que tu código no mejora nada (Hipótesis Nula) hasta que los datos demuestren lo contrario.

El Stack de Ejecución del Prototipo



¿Es atractivo?
Diseño visual y comunicación de la jerarquía.

¿Es fácil?
Reducción de fricción, modelos mentales.

¿Es posible?
Código semántico, carga rápida, compatibilidad.

⚠

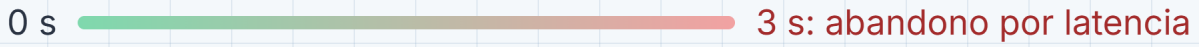
Si la accesibilidad o usabilidad fallan, nunca sabrás si falló tu hipótesis original o si tu código simplemente estuvo mal implementado.

Capa 1: Accesibilidad (El Cimiento Técnico)

Si el usuario no puede entrar o interactuar, la idea muere instantáneamente.

```
<!— ERROR TÍPICO —>
<div onClick='unirse()'>Unirse</div>
<!— CÓDIGO SEMÁNTICO (CORRECTO) —>
<button type='button'>Unirse</button>
<!— FALTA EN EL FUNKY —>
<div aria-live='polite'>…</div>
```

- **Rendimiento:** Tiempos de carga menores a 2 segundos (el exceso de JavaScript arruina prototipos). Meta de clase; aún sin medir en el Funky.
- **Código Semántico:** Etiquetas HTML nativas para lectores de pantalla y teclado. Unirse ya es un <button>; falta aria-live para anunciar la presencia.
- **Localización Técnica:** Formateo de fechas e idiomas según el navegador (ej. es-MX vs en-US).

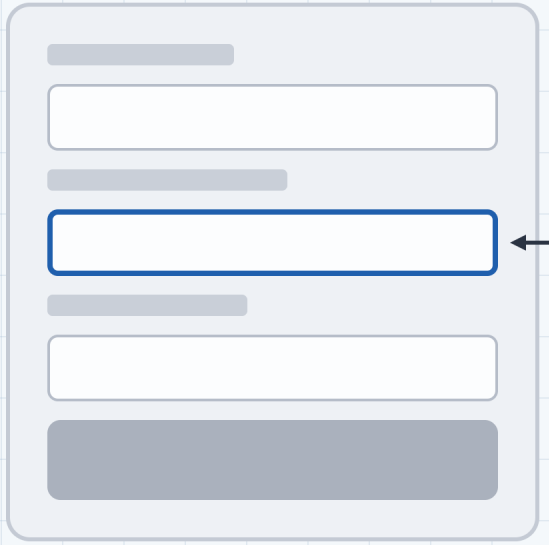


Capa 2: Usabilidad (Reducción de Fricción)

Interfaces intuitivas que no requieren esfuerzo cognitivo.

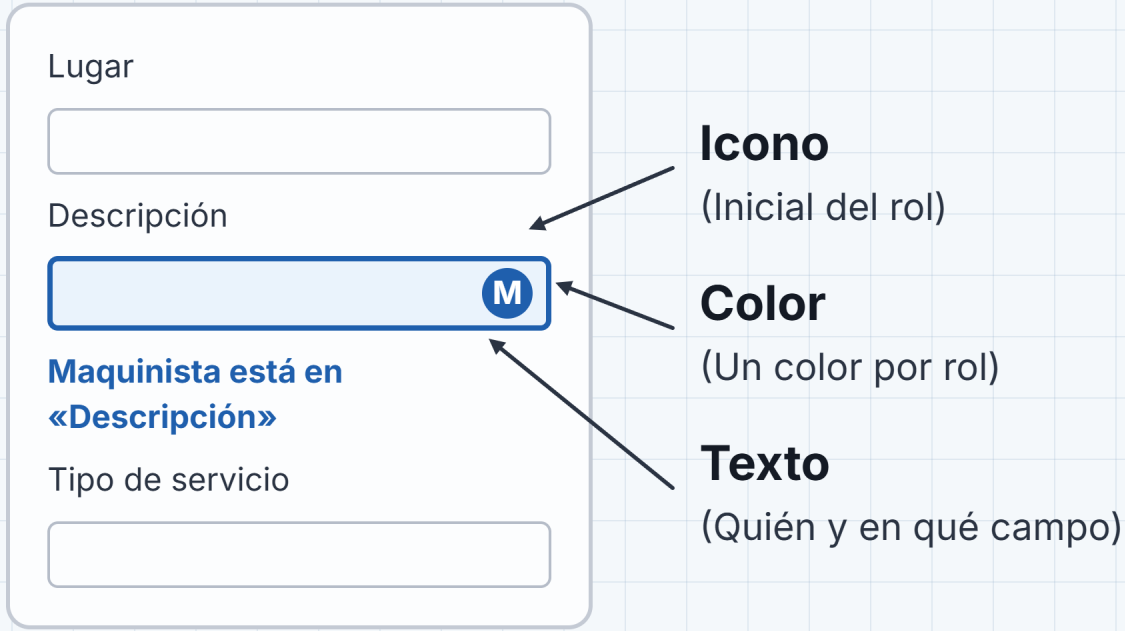
Ley de Jakob: Los usuarios pasan la mayor parte de su tiempo en otros sitios. Tu prototipo debe respetar los modelos mentales estándar.

Carga Cognitiva Alta



Depende de un solo indicador visual (Color).

Diseño sin Fricción (Redundancia)



Capa 3: Estética (El Diseño como Información)

La forma sigue a la función. El diseño visual no es adorno, es una herramienta para guiar el flujo del sistema.

Jerarquía Plana (Fallo de diseño)

Parte en curso

Tu jefe de turno abrió el parte de esta salida.

Unirse

Contraste Funcional (Guía el ojo)

Parte en curso

Tu jefe de turno abrió el parte de esta salida.

Unirse

Atracción secundaria
(Contexto)

Foco principal
(Acción: Unirse)

El Motor del Comportamiento: El Modelo Fogg

Para que el prototipo funcione, las tres variables deben converger al mismo tiempo.

B (Comportamiento)

Que los 3 llenen un solo parte entre todos.

$$B = M \times A \times P$$

M (Motivación)

El deseo del usuario (alta o baja).

Nuestro caso: no duplicar datos durante el trayecto.

A (Habilidad)

Qué tan fácil es técnicamente ejecutar la acción.

Nuestro caso: cada quien llena su parte desde su celular.

P (Prompt / Gatillo)

El detonante o interfaz que pide la acción.

Nuestro caso: la invitación «Parte en curso» con el botón Unirse.

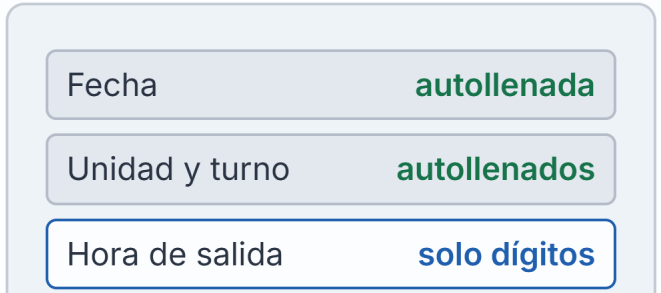
B = M × A × P Traducido a Código y UI

M (Motivación)



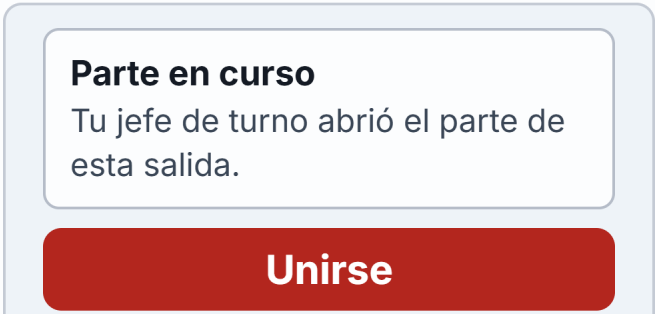
- Copy claro: «Entre los 3 llenan el parte».
- Reparto del trabajo: unos 30 campos entre 3 son ~7 por persona (estimación del proyecto).
- Obligatorios a la vista: C-5, tipo de servicio, lugar, descripción y 1 foto.

A (Habilidad)



- Autollenado de fecha, unidad, turno, estación y personal.
- Horas con teclado numérico: solo dígitos.
- Presencia compartida: cada quien ve dónde está el otro, sin bloqueos.

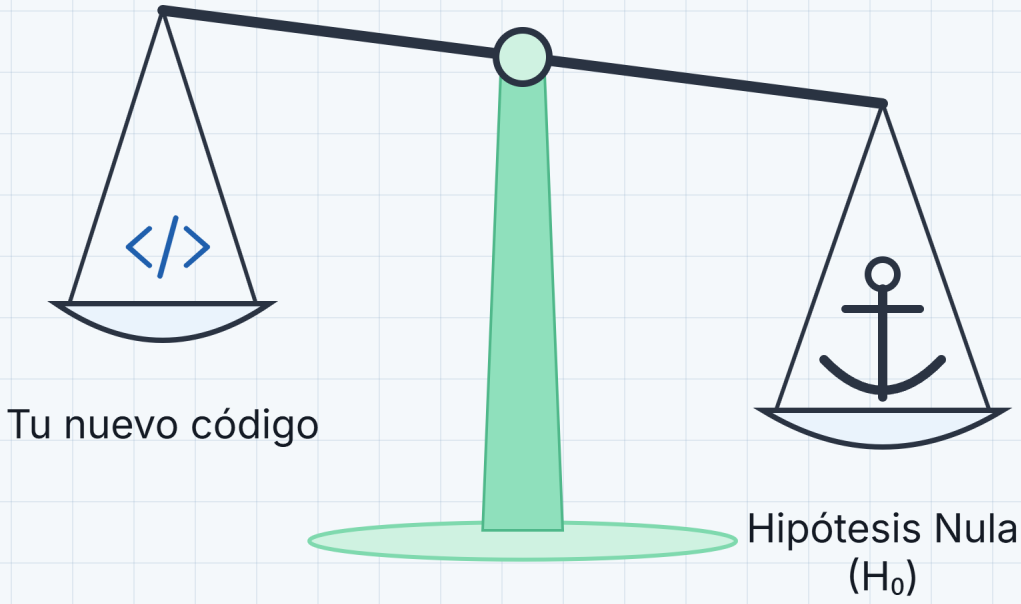
P (Prompt)



- Invitación «Parte en curso» al abrir la app.
- Botón Unirse de rojo sólido, alto contraste y ancho completo.
- Solo el jefe de turno envía el parte al final.

Prueba como si estuvieras equivocado

Escepticismo Radical y la Hipótesis Nula (H_0)



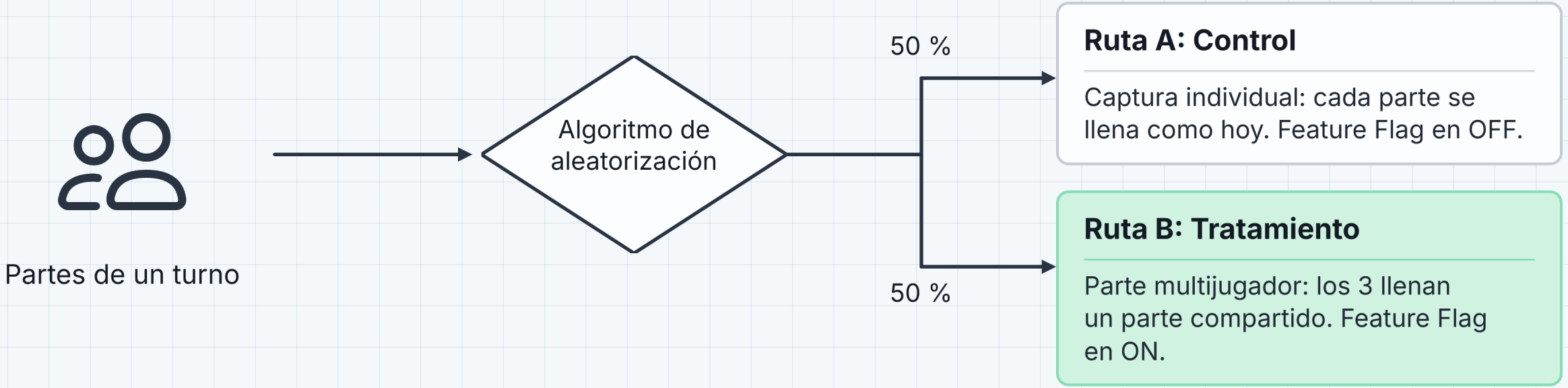
- En la ciencia de datos, la Hipótesis Nula asume que el cambio que programaste **NO** tiene ningún efecto en el comportamiento del usuario.
- No buscamos «probar que nuestra idea es buena». Buscamos datos estadísticamente significativos que nos obliguen a rechazar la Hipótesis Nula.
- Si asumes que tu código es un fracaso desde el inicio, evitas el sesgo de confirmación y evalúas la realidad.

NUESTRO CASO

H_0: el parte multijugador no cambia el tiempo activo de captura.

H_1: lo reduce al menos 20 % en la mediana (umbral propuesto, aún sin línea base).

El Experimento Aleatorio Controlado (Pruebas A/B)



Concepto clave: La aleatorización neutraliza las variables de confusión (tipo de servicio, hora del día, qué personal está de guardia). **La ÚNICA diferencia real entre ambos grupos es el cambio en tu código** (la variable independiente): el multijugador.

Unidad de aleatorización: el parte completo. Los 3 celulares de una unidad caen en el mismo grupo.

La Regla de Decisión

Rechazamos H_0 solo si se cumplen las tres condiciones.

1

Significancia

$p < 0.05$ al comparar las medianas de A y B (prueba Mann-Whitney U).

Se usa la mediana para que unos pocos partes larguísimos no muevan el resultado.

2

Efecto mínimo

B baja la mediana del tiempo activo al menos 20 % frente a A.

Umbral propuesto por el equipo: aún no hay línea base medida.

3

Guardarraíles

Sin empeorar: partes completos, errores de envío y datos perdidos.

Un parte más rápido pero incompleto no es una mejora.

Métrica: mediana del tiempo activo del parte completo, de «Nuevo parte» a «Enviar».

Muestra mínima (propuesta): 30 partes por grupo, 2 semanas o más.

Infraestructura Científica: Feature Flags



Desplegar ≠ Lanzar

Puedes subir código a producción (deploy) sin que los usuarios lo vean (release).

Envoltorio lógico

Los Feature Flags envuelven tu prototipo en un condicional lógico, activándolo a voluntad.

Beneficio científico

Permiten aislar variables habilitando el prototipo solo para la prueba, y apagarlo al instante (Kill Switch) si ocurre un error crítico.

Nuestro Plan de Feature Flag

Parte multijugador: cuándo se enciende y cuándo se apaga.

```
// corre sobre el Functional Prototype (hito 4), no sobre el Funky
feature_flag:    "enable_parte_multijugador"
activar_100:     "se rechaza H₀ y los guardarraíles no empeoran"
kill_switch:     "dato perdido, errores de envío > 1 % (propuesta)
                  o jefe sin poder enviar"
// al apagar: todo vuelve al parte individual y el parte en curso
se conserva
```

⚠

Para correr este experimento necesitamos un servidor que guarde los partes y asigne el grupo. Hoy el Funky no lo tiene: llega con el hito 4 (Node, Express y MySQL).

1. Desplegar

Código en producción con el flag en OFF.

→

2. Experimentar

Flag al 50 %, por parte, durante 2 semanas o más.

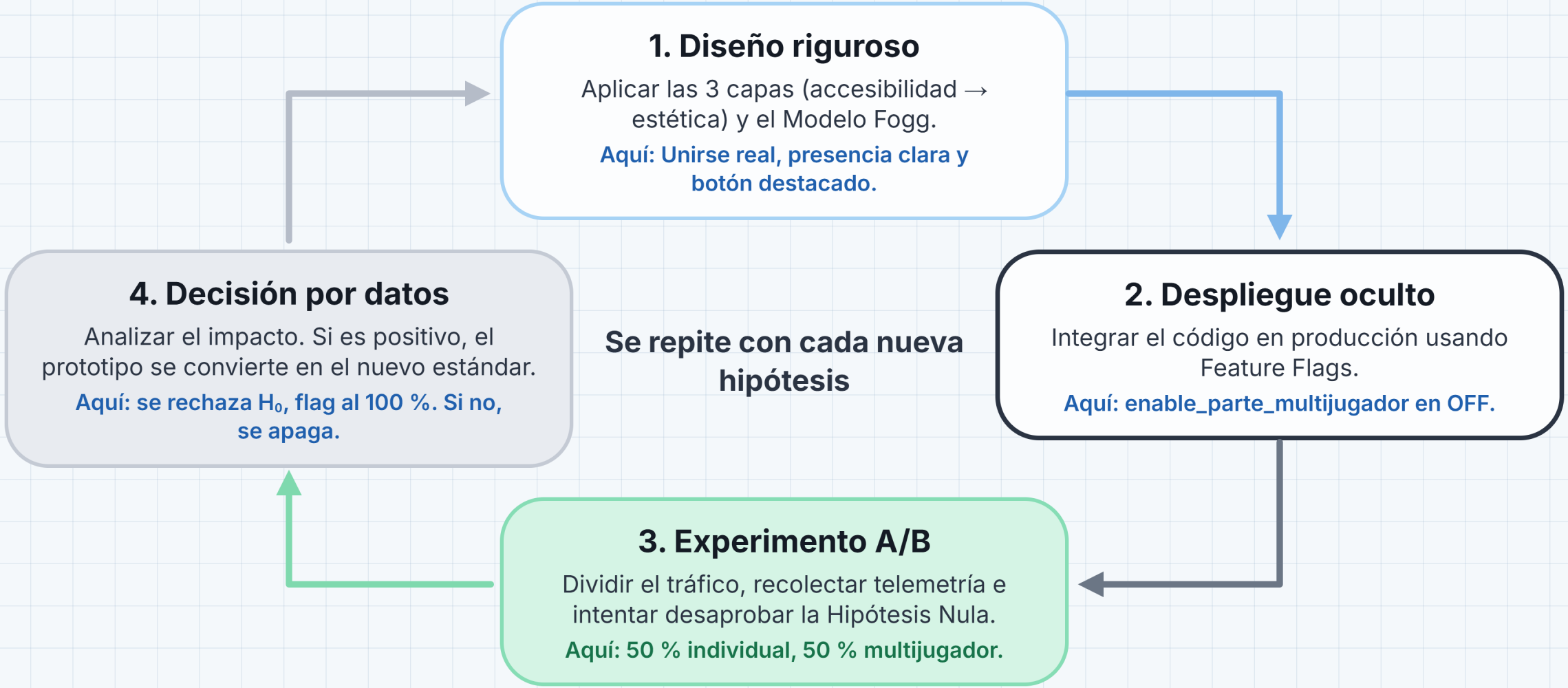
→

3. Decidir

Activar al 100 % o apagar el flag y aprender.

UABC

El Bucle Científico de Producto



El Ingeniero Moderno

El código es la hipótesis.

Un prototipo ya no es un boceto, es un cambio funcional inyectado en un ecosistema vivo.

La ejecución aísla la variable.

Debes programar y diseñar con perfección (Accesibilidad → Estética) para que los datos reflejen el valor de la idea, no los errores técnicos.

Científicos del comportamiento.

Escribir software moderno no se trata solo de que el código compile; se trata de medir el impacto humano a través del método científico.

En nuestro proyecto: el parte multijugador es una hipótesis hasta que los datos nos obliguen a rechazar H_0.